Sesión 8

Captura de Datos y Extracción Masiva

Carlos César Sánchez Coronel

BASES DE DATOS

Contenido general

1. Introducción a la Captura de Datos
2. Fuentes Externas y Web Scraping
3. APIs (Application Programming Interfaces)
4. Captura en Tiempo Real y Streaming
5. Formatos de Datos y Almacenamiento
6. Procesamiento a Gran Escala
7. Ejemplo Integrador: Captura de Tweets en Tiempo Real
8. Ejercicios Resueltos
9. Glosario

Contenido de la sección

1. Introducción a la Captura de Datos
2. Fuentes Externas y Web Scraping
3. APIs (Application Programming Interfaces)
4. Captura en Tiempo Real y Streaming
5. Formatos de Datos y Almacenamiento
6. Procesamiento a Gran Escala
7. Ejemplo Integrador: Captura de Tweets en Tiempo Real
8. Ejercicios Resueltos
9. Glosario

Importancia de la Captura de Datos

- **Alimentación de modelos de IA**: Los modelos requieren grandes volúmenes de datos diversos (texto, imágenes, sensores).
- **Toma de decisiones en tiempo real**: Fraude, recomendaciones, monitoreo.
- **Análisis histórico**: Tendencias, estudios retrospectivos.
- **Integración de sistemas**: Unificar fuentes heterogéneas.

Clasificación de Fuentes de Datos

Según origen

- **Internas**: BD transaccionales, logs, archivos corporativos.
- **Externas**: Web, APIs, datos gubernamentales, redes sociales.

Según forma de entrega

- **Batch**: Datos en bloques periódicos (diario, semanal).
- **Streaming**: Datos continuos en tiempo real (eventos, sensores).

Contenido de la sección

1. Introducción a la Captura de Datos
2. Fuentes Externas y Web Scraping
3. APIs (Application Programming Interfaces)
4. Captura en Tiempo Real y Streaming
5. Formatos de Datos y Almacenamiento
6. Procesamiento a Gran Escala
7. Ejemplo Integrador: Captura de Tweets en Tiempo Real
8. Ejercicios Resueltos
9. Glosario

Web Scraping

Definición

Extracción automatizada de información de sitios web.

Casos de uso

- Entrenamiento de LLMs (Common Crawl).
- Monitoreo de precios de la competencia.
- Agregadores de noticias.
- Investigación académica.

Herramientas de Scraping en Python

- **requests** Peticiones HTTP.
- **BeautifulSoup** Parseo de HTML.
- **Selenium** Automatización de navegadores (JavaScript).
- **Scrapy** Framework para scraping a gran escala.

Ejemplo básico

```python
import requests
from bs4 import BeautifulSoup

url = 'https://quotes.toscrape.com/'
response = requests.get(url)
soup = BeautifulSoup(response.text, 'html.parser')

for quote in soup.select('.quote'):
    text = quote.select_one('.text').get_text()
    author = quote.select_one('.author').get_text()
    print(f'{author}: {text}')
```

Consideraciones Legales y Éticas

- Respetar `robots.txt`.
- No sobrecargar servidores (rate limiting, delays).
- Verificar términos de servicio.
- Cumplir normativas de protección de datos (GDPR, CCPA).

Common Crawl

¿Qué es?

Organización sin fines de lucro que realiza rastreos masivos de la web y pone los datos a disposición pública (petabytes en formato WARC).

Importancia para IA

Base de entrenamiento de modelos como GPT, BERT, etc. por su diversidad y escala.

Contenido de la sección

1. Introducción a la Captura de Datos
2. Fuentes Externas y Web Scraping
3. APIs (Application Programming Interfaces)
4. Captura en Tiempo Real y Streaming
5. Formatos de Datos y Almacenamiento
6. Procesamiento a Gran Escala
7. Ejemplo Integrador: Captura de Tweets en Tiempo Real
8. Ejercicios Resueltos
9. Glosario

Tipos de APIs

REST	Basada en HTTP, verbos (GET, POST), devuelve JSON/XML.
GraphQL	Cliente especifica campos necesarios.
SOAP	Protocolo pesado basado en XML, entornos empresariales.
WebSockets	Comunicación bidireccional en tiempo real.

Autenticación y Rate Limiting

- **API Key** en header o parámetro.
- **OAuth** tokens.
- **Rate limiting**: límite de peticiones por tiempo; respetar para no ser bloqueado.

Ejemplo con API REST (JSONPlaceholder)

```python
import requests
import json

url = 'https://jsonplaceholder.typicode.com/posts'
response = requests.get(url)
if response.status_code == 200:
    posts = response.json()
    with open('posts.json', 'w') as f:
        json.dump(posts, f, indent=2)
else:
    print('Error:', response.status_code)
```

Contenido de la sección

1. Introducción a la Captura de Datos
2. Fuentes Externas y Web Scraping
3. APIs (Application Programming Interfaces)
4. Captura en Tiempo Real y Streaming
5. Formatos de Datos y Almacenamiento
6. Procesamiento a Gran Escala
7. Ejemplo Integrador: Captura de Tweets en Tiempo Real
8. Ejercicios Resueltos
9. Glosario

Concepto de Streaming

Procesamiento continuo de datos a medida que llegan

- Baja latencia (ms).
- Eventos, sensores, logs, redes sociales.

Casos de uso

- Detección de fraude en transacciones.
- Monitoreo IoT.
- Análisis de tendencias en redes sociales.

Change Data Capture (CDC)

Captura de cambios en bases de datos en tiempo real

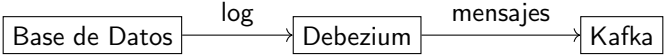
- Inserciones, actualizaciones, eliminaciones.
- Sin afectar el rendimiento transaccional.

Basado en logs Lee log transaccional (WAL, binlog). Ej: Debezium.

Basado en triggers Impacta rendimiento.

Consultas periódicas No es tiempo real.

Debezium y Kafka



- Debezium se conecta a la BD y publica cambios en tópicos Kafka.
- Consumidores (Spark, aplicaciones) procesan los eventos.

Apache Kafka

Plataforma de streaming distribuida

- **Tópicos**: categorías de mensajes.
- **Productores**: publican mensajes.
- **Consumidores**: leen mensajes.
- **Partitions**: paralelismo y orden.

Ventajas

Alta escalabilidad, persistencia, tolerancia a fallos, procesamiento en tiempo real.

Otras Plataformas de Streaming

- **AWS Kinesis**: servicio gestionado en AWS.
- **Google Pub/Sub**: global, push/pull.
- **Apache Pulsar**: almacenamiento en capas.
- **RabbitMQ**: orientado a colas (menos escalable).

Contenido de la sección

1. Introducción a la Captura de Datos
2. Fuentes Externas y Web Scraping
3. APIs (Application Programming Interfaces)
4. Captura en Tiempo Real y Streaming
5. Formatos de Datos y Almacenamiento
6. Procesamiento a Gran Escala
7. Ejemplo Integrador: Captura de Tweets en Tiempo Real
8. Ejercicios Resueltos
9. Glosario

Formatos de Archivo

Formato	Tipo	Compresión	Uso típico
CSV	Texto fila	Baja	Intercambio simple
JSON	Texto semi-estructurado	Media	APIs, documentos
Avro	Binario fila con esquema	Alta	Streaming (Kafka)
Parquet	Columnar	Alta	Análisis, data lakes
ORC	Columnar	Alta	Hive, Hadoop
WARC	Archivo web	Variable	Common Crawl

Data Lakes vs Data Warehouses

Data Lake

- Almacena datos en bruto (formatos nativos).
- Flexible, estructurados y no estructurados.
- Ej: Amazon S3, Azure Data Lake, HDFS.

Data Warehouse

- Datos procesados y estructurados (esquema estrella).
- Optimizado para consultas analíticas rápidas.
- Ej: Snowflake, BigQuery, Redshift.

Particionamiento en Data Lakes

Organización en carpetas

```
s3://bucket/ventas/year=2025/month=03/day=15/ventas.parquet
```

- Permite a motores como Spark leer solo las particiones necesarias.
- Reduce E/S y acelera consultas.

Contenido de la sección

1. Introducción a la Captura de Datos
2. Fuentes Externas y Web Scraping
3. APIs (Application Programming Interfaces)
4. Captura en Tiempo Real y Streaming
5. Formatos de Datos y Almacenamiento
6. Procesamiento a Gran Escala
7. Ejemplo Integrador: Captura de Tweets en Tiempo Real
8. Ejercicios Resueltos
9. Glosario

Apache Spark

Motor unificado de procesamiento distribuido

- **Spark SQL**: consultas SQL sobre DataFrames.
- **DataFrames**: API similar a tablas.
- **Structured Streaming**: procesamiento en streaming.
- **MLlib**: machine learning a escala.

Ejemplo con PySpark

Leer CSV, calcular total por producto y guardar en Parquet

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ventas").getOrCreate()

df = spark.read.option("header", "true").csv("ruta/ventas.csv")
df = df.withColumn("monto", df["cantidad"].cast("int") * df["precio"].cast("float"))
resultado = df.groupBy("producto").sum("monto").withColumnRenamed("sum(monto)", "total")
resultado.write.parquet("resultado.parquet")
```

Apache Flink vs Spark Streaming

Spark Streaming Micro-batches (intervalos de tiempo). Robusto, maduro.

Flink Streaming puro, evento por evento, menor latencia. Ideal para fraudes, monitoreo.

Hadoop MapReduce

- Modelo de programación distribuida (map y reduce).
- Lento porque escribe resultados intermedios en disco.
- Reemplazado por Spark en la mayoría de casos.

Contenido de la sección

1. Introducción a la Captura de Datos
2. Fuentes Externas y Web Scraping
3. APIs (Application Programming Interfaces)
4. Captura en Tiempo Real y Streaming
5. Formatos de Datos y Almacenamiento
6. Procesamiento a Gran Escala
7. Ejemplo Integrador: Captura de Tweets en Tiempo Real
8. Ejercicios Resueltos
9. Glosario

Arquitectura del Pipeline

1. **Fuente**: API de Twitter (streaming) con autenticación OAuth.
2. **Ingesta**: Productor Kafka (Python) publica tweets en tópico `tweets`.
3. **Procesamiento**: Spark Structured Streaming lee de Kafka, limpia (quita RTs), extrae hashtags, y guarda en Parquet particionado por fecha.
4. **Almacenamiento**: Data lake en S3 (Parquet).
5. **Análisis batch**: Spark genera reportes diarios.

Productor Kafka en Python

```
from kafka import KafkaProducer
import tweepy, json

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

class StreamListener(tweepy.Stream):
    def on_data(self, raw_data):
        tweet = json.loads(raw_data)
        producer.send('tweets', value=tweet)
        return True

stream = StreamListener(consumer_key, consumer_secret, access_token, access_token_secret)
stream.filter(track=['#datascience'])
```

Consumidor Spark Structured Streaming

```python
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructType, StructField, StringType, TimestampType

spark = SparkSession.builder.appName("tweet_analysis").getOrCreate()

schema = StructType([
    StructField("id", StringType()),
    StructField("text", StringType()),
    StructField("created_at", TimestampType())
])

df = spark.readStream.format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "tweets") \
    .load() \
    .select(from_json(col("value").cast("string"), schema).alias("data")) \
    .select("data.*")

# Limpieza: quitar retweets
df_clean = df.filter(~col("text").startswith("RT"))

# Escribir en Parquet particionado por fecha
query = df_clean.writeStream \
    .format("parquet") \
    .partitionBy("created_at") \
    .option("path", "s3a://bucket/tweets/") \
```

Contenido de la sección

1. Introducción a la Captura de Datos
2. Fuentes Externas y Web Scraping
3. APIs (Application Programming Interfaces)
4. Captura en Tiempo Real y Streaming
5. Formatos de Datos y Almacenamiento
6. Procesamiento a Gran Escala
7. Ejemplo Integrador: Captura de Tweets en Tiempo Real
8. Ejercicios Resueltos
9. Glosario

Ejercicio 1: Scraping de una Página Web

Enunciado: Extraer citas, autores y etiquetas de `http://quotes.toscrape.com` y guardar en CSV.

Solución

```python
import requests, csv
from bs4 import BeautifulSoup

url = 'http://quotes.toscrape.com'
quotes = []
while url:
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')
    for quote in soup.select('.quote'):
        text = quote.select_one('.text').get_text()
        author = quote.select_one('.author').get_text()
        tags = [tag.get_text() for tag in quote.select('.tag')]
        quotes.append([text, author, ', '.join(tags)])
    next_btn = soup.select_one('.next a')
    url = next_btn['href'] if next_btn else None

with open('quotes.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.writer(f)
    writer.writerow(['quote', 'author', 'tags'])
    writer.writerows(quotes)
```

Ejercicio 2: Consumo de API Pública

Enunciado: Obtener posts de JSONPlaceholder y guardarlos en JSON.

Solución

```python
import requests, json

response = requests.get('https://jsonplaceholder.typicode.com/posts')
posts = response.json()
with open('posts.json', 'w') as f:
    json.dump(posts, f, indent=2)
```

Ejercicio 3: Simulación de CDC con Triggers en PostgreSQL

Enunciado: Crear trigger que registre cambios en `clientes` y un script Python que lea la auditoría y publique en Kafka.

Trigger (PostgreSQL)

```sql
CREATE TABLE clientes_audit (...);
CREATE OR REPLACE FUNCTION audit_clientes() RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO clientes_audit (cliente_id, nombre_nuevo, email_nuevo, operacion)
        VALUES (NEW.id, NEW.nombre, NEW.email, 'I');
    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO clientes_audit (cliente_id, nombre_anterior, nombre_nuevo, email_anterior,
            email_nuevo, operacion)
        VALUES (OLD.id, OLD.nombre, NEW.nombre, OLD.email, NEW.email, 'U');
    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO clientes_audit (cliente_id, nombre_anterior, email_anterior, operacion)
        VALUES (OLD.id, OLD.nombre, OLD.email, 'D');
    END IF;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

Script Python para leer auditoría y enviar a Kafka

```
import psycopg2
from kafka import KafkaProducer
import json

conn = psycopg2.connect("dbname=test user=postgres")
cur = conn.cursor()
cur.execute("SELECT * FROM clientes_audit WHERE fecha > NOW() - INTERVAL '1 minute'")
rows = cur.fetchall()

producer = KafkaProducer(bootstrap_servers='localhost:9092',
                         value_serializer=lambda v: json.dumps(v).encode('utf-8'))

for row in rows:
    data = {'id': row[0], 'cliente_id': row[1], 'operacion': row[7], 'fecha': str(row[6])}
    producer.send('clientes_audit', value=data)
```

Ejercicio 4: Procesamiento Batch con Spark

Enunciado: Cargar CSV de ventas, calcular total por producto y guardar en Parquet.

Solución

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("ventas").getOrCreate()
df = spark.read.option("header", "true").csv("ruta/ventas*.csv")
df = df.withColumn("monto", df["cantidad"].cast("int") * df["precio"].cast("float"))
resultado = df.groupBy("producto").sum("monto").withColumnRenamed("sum(monto)", "total")
resultado.write.parquet("resultado.parquet")
```

Contenido de la sección

1. Introducción a la Captura de Datos
2. Fuentes Externas y Web Scraping
3. APIs (Application Programming Interfaces)
4. Captura en Tiempo Real y Streaming
5. Formatos de Datos y Almacenamiento
6. Procesamiento a Gran Escala
7. Ejemplo Integrador: Captura de Tweets en Tiempo Real
8. Ejercicios Resueltos
9. Glosario

Términos Clave

Web Scraping	Extracción automatizada de datos web.
API	Interfaz para comunicación entre sistemas.
CDC	Change Data Capture, captura de cambios en BD.
Streaming	Procesamiento continuo de datos.
Kafka	Plataforma de streaming distribuida.
Debezium	Herramienta de CDC sobre Kafka.
Parquet	Formato columnar optimizado para análisis.
Data Lake	Almacén de datos en bruto.
Spark	Motor de procesamiento distribuido.

¡Gracias!